บทที่ 10

การใช้งานคลาวด์ดาต้าเบสด้วย Supabase

การพัฒนาแอปพลิเคชันที่มีระบบฐานข้อมูลหลังบ้านแบบเรียลไทม์ โดยไม่ต้องเสียเวลาเขียนโค้ดและตั้งค่าเครื่องเซิร์ฟเวอร์เอง ถือเป็นความสะดวกที่ทำได้จริงผ่านระบบ **Backend-as-a-Service (BaaS)** ในบทเรียนนี้เราจะศึกษาขั้นตอนการเชื่อมโยงแอปพลิเคชัน Expo กับ **Supabase Cloud Database** ซึ่งขับเคลื่อนด้วย PostgreSQL และเรียนรู้วิธีจัดการฐานข้อมูลครบวงจร (CRUD Operations) ทั้งการเพิ่ม ดึง แก้ไข และลบข้อมูล

10.1 สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase

Supabase เป็นแพลตฟอร์มหลังบ้านสำเร็จรูปที่เป็นมิตรต่อผู้พัฒนาและเปิดใช้งานแบบ Open-source (Supabase, 2024) โดยมีโครงสร้างฐานข้อมูลหลักเป็น **PostgreSQL** ซึ่งมีความเสถียรและรองรับความสัมพันธ์ของข้อมูล (Relational Database)

ทำไมต้องเลือก Supabase?

• **พร้อมใช้งานทันที:** มีระบบ Database, Auth, และ Storage มาให้พร้อมใช้โดยไม่ต้องติดตั้งและตั้งค่าระบบเครือข่ายเซิร์ฟเวอร์เอง

• **รองรับระบบ SQL แท้:** สามารถเขียน Query จัดความสัมพันธ์ข้อมูล หรือประกาศ Constraints ป้องกันตารางพังได้เหมือนฐานข้อมูลสากลทั่วไป

• **ประสิทธิภาพระดับเนทีฟ:** ไลบรารีพัฒนามาให้รองรับระบบ Asynchronous (Promise-based) ทำงานร่วมกับ React Native ได้อย่างไร้รอยต่อ

ขั้นตอนการเตรียมคลาวด์โปรเจกต์:

1. ลงทะเบียนและล็อกอินเข้าใช้งานที่ supabase.com
2. สร้างโครงการใหม่ (Create New Project) และระบุรหัสผ่านฐานข้อมูล
3. ไปที่เมนู **Project Settings > API** ของแผงควบคุมหลังบ้าน เพื่อคัดลอกค่าความปลอดภัยหลัก 2 คีย์ ได้แก่:

• **Project URL**: ที่อยู่โดเมนชี้เฉพาะสำหรับส่ง API รีเควส

• **Anon Key (Public)**: คีย์สาธารณะสำหรับสแกนส่งสิทธิ์อนุญาตเข้าถึงตารางข้อมูล

10.2 การติดตั้งและตั้งค่า Supabase Client ใน Expo (SDK 54)

เพื่อเชื่อมแอปพลิเคชัน Expo เข้าหาตารางฐานข้อมูล เราจะติดตั้งชุดซอฟต์แวร์พัฒนาอย่างเป็นทางการร่วมกับตัวช่วยจัดระเบียบ URL สำหรับ React Native (Supabase, 2024):

```
npm install @supabase/supabase-js react-native-url-polyfill
```

หมายเหตุ: `react-native-url-polyfill` ทำหน้าที่เป็นสะพานเชื่อมฟังก์ชันการประมวลผลคำสั่ง URL ของตัวเนทีฟเครื่องให้สอดรับกับมาตรฐานเว็บ เนื่องจาก React Native รันไทม์ไม่มีฟังก์ชัน URL ดั้งเดิม

การตั้งค่าตัวแปรสภาพแวดล้อม (Environment Variables) ใน SDK 54:

เพื่อความปลอดภัย ห้ามนำเอาคีย์ความลับอย่าง Anon Key ไปแปะเขียนโค้ดลงไปตรง ๆ แต่ให้ใช้ไฟล์ **`.env`** ในโฟลเดอร์นอกสุดของโปรเจกต์:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-project-id.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

หมายเหตุ: ใน Expo SDK 54 ตัวแปรสภาพแวดล้อมที่ขึ้นต้นด้วย `EXPO_PUBLIC_` จะถูกแปลงและโหลดเข้าสู่ระบบรันไทม์จาวาสคริปต์อัตโนมัติ

การเขียนไฟล์ตั้งค่าเชื่อมโยง client (`lib/supabase.ts`):

```
import 'react-native-url-polyfill/auto'; // นำเข้าเพื่อแก้บั๊กคำสั่ง URL ในระบบเนทีฟ
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

// สร้างและส่งออก supabase object ไปใช้งานข้ามไฟล์
export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

10.3 การอ่านข้อมูลจากตาราง (Read / Select Data)

การสืบค้นดึงข้อมูลแถวตารางจากฐานข้อมูล Supabase จะทำผ่านเมธอด **`.select()`** โดยสกัดผลลัพธ์ข้อมูลออกมาในคีย์ `data` และตรวจจับปัญหาการอ่านในคีย์ `error`

```
import { supabase } from '../lib/supabase';

interface Task {
  id: number;
```

```
  title: string;
  is_completed: boolean;
  created_at: string;
}

const fetchTasks = async (): Promise<Task[]> => {
  const { data, error } = await supabase
  .from('tasks') // อ้างอิงชื่อตารางใน Supabase
  .select('*')   // ดึงฟิลด์ข้อมูลทุกคอลัมน์
  .order('created_at', { ascending: false }); // เรียงลำดับเอาโพสต์ใหม่ขึ้นก่อน

  if (error) {
    console.error('ดึงข้อมูลล้มเหลว:', error.message);
    return [];
  }

  return data as Task[];
};
```

การใส่ตัวกรอง (Query Filters):

เราสามารถคัดกรองข้อมูลเฉพาะแถวที่ต้องการได้ด้วยการต่อเมธอด เช่น:

- `.eq('is_completed', true)`: แสดงเฉพาะโพสต์ที่เสร็จสมบูรณ์แล้ว (Equal)
- `.ilike('title', '%เรียน%')`: ค้นหาชื่อโพสต์ที่มีตัวอักษร "เรียน" ผสมอยู่โดยไม่สนใจตัวพิมพ์เล็กใหญ่

10.4 การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data)

ในการสร้างแถวข้อมูลชุดใหม่ (เช่น การลงทะเบียนหรือบันทึกสินค้าเพิ่ม) จะส่งผ่านเมธอด `.insert()` โดยยอมรับข้อมูลในรูปแบบของอาร์เรย์ออบเจกต์ (Array of Objects)

```
const createNewTask = async (taskTitle: string) => {
  const { data, error } = await supabase
  .from('tasks')
  .insert([
      {
        title: taskTitle,
        is_completed: false
      }
  ])
  .select(); // บอกให้ Supabase ส่งข้อมูลแถวล่าสุดที่สร้างสำเร็จกลับคืนมาด้วย

  if (error) {
    console.error('บันทึกไม่สำเร็จ:', error.message);
```

```
    return null;
  }

  return data[0]; // คืนค่าออบเจกต์ข้อมูลชิ้นแรกที่พึ่งบันทึกสำเร็จ
};
```

10.5 การแก้ไขและปรับปรุงข้อมูล (Update Data)

เมื่อมีการคลิกทำเครื่องหมายติ๊กถูก หรือต้องการเปลี่ยนข้อความแถวข้อมูล เราจะใช้เมธอด `.update()` โดยต้องระบุเงื่อนไขเสมอด้วย `.eq()` เพื่อป้องกันไม่ให้คำสั่งเข้าไปทับข้อมูลของแถวอื่น ๆ ทั้งตาราง

```
const toggleTaskCompletion = async (taskId: number, currentStatus:
boolean) => {
  const { data, error } = await supabase
  .from('tasks')
  .update({
      is_completed: !currentStatus
    }) // ระบุคอลัมน์และค่าใหม่ที่ต้องการแก้ไข
  .eq('id', taskId) // กรองอัปเดตเฉพาะแถวที่มี ID ตรงกัน
  .select();

  if (error) {
    console.error('อัปเดตข้อมูลล้มเหลว:', error.message);
    return null;
  }

  return data[0];
};
```

10.6 การลบข้อมูลออกจากตาราง (Delete Data)

การสั่งถอดถอนแถวข้อมูลออกจากสาระบบบอย่างถาวร จะทำผ่านเมธอด `.delete()` โดยต้องผูกตัวกรอง .eq() ชี้เฉพาะเจาะจงเป้าหมาย

```
const deleteTask = async (taskId: number): Promise<boolean> => {
  const { error } = await supabase
  .from('tasks')
  .delete()
  .eq('id', taskId); // ลบเฉพาะแถวที่มี ID ตรงกันเท่านั้น
```

```
  if (error) {
    console.error('ลบข้อมูลไม่สำเร็จ:', error.message);
    return false;
  }

  return true; // ส่งค่าความสำเร็จกลับไปอัปเดตสถานะลิสต์บนหน้าจอแอป
};
```

10.7 การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด (Loading & Error Handling)

จุดแตกต่างของการติดต่อฐานข้อมูลตรงบนคลาวด์เมื่อเทียบกับการเก็บตัวแปรบนเครื่อง คือความเร็วการตอบรับจะแปรผันตามระยะทางและคุณภาพของเครือข่ายอินเทอร์เน็ต ผู้พัฒนาต้องเตรียมกลไกแจ้งสถานะอย่างมีชั้นเชิง:

```
import React, { useState, useEffect } from 'react';
import { View, Text, ActivityIndicator, Alert, Button } from 'react-
native';
import { supabase } from '../lib/supabase';

export default function LoadingExample() {
  const [tasks, setTasks] = useState<any[]>([]);
  const [loading, setLoading] = useState(true);

  const loadData = async () => {
    try {
      setLoading(true);
      const { data, error } = await supabase.from('tasks').select('*');

      if (error) {
        throw error; // ดีดเข้าบล็อก catch เพื่อแยกส่วนจัดเตือน
      }

      setTasks(data || []);
    } catch (error: any) {
      Alert.alert('เชื่อมโยงฐานข้อมูลขัดข้อง', error.message || 'กรุณาลองใหม่อีกครั้ง');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    loadData();
  }, []);
```

```
  if (loading) {
    return (
      <View style={{ flex: 1, justifyContent: 'center', alignItems:
'center' }}>
        <ActivityIndicator size="large" color="#38bdf8" />
        <Text style={{ marginTop: 8, color: '#64748b' }}>กำลังเชื่อมต่อฐานข้อมูลระบบ
คลาวด์...</Text>
      </View>
  );
  }

  return (
    <View style={{ padding: 20 }}>
      <Text style={{ fontSize: 18, fontWeight: 'bold' }}>จำนวนงานคงค้าง:
{tasks.length} รายการ</Text>
      <Button title="🔄 ดึงข้อมูลสดใหม่" onPress={loadData} />
    </View>
 );
}
```

สรุปท้ายบทเรียน

สถาปัตยกรรม BaaS และคลาวด์ดาต้าเบสของ Supabase: Supabase เป็นระบบหลังบ้านสำเร็จรูปประเภท Backend-as-a-Service (BaaS) ที่ขับเคลื่อนด้วยฐานข้อมูล PostgreSQL ทำให้ประยุกต์จัดความสัมพันธ์ของข้อมูลแบบ SQL แท้ได้อย่างเต็มรูปแบบ โดยผู้พัฒนาสามารถเริ่มใช้งานได้ทันทีและเข้าถึงคีย์ความปลอดภัยอย่าง Project URL และ Anon Key เพื่อเข้าสิทธิ์เชื่อมต่อการทำงานต่าง ๆ

การติดตั้งและตั้งค่า Supabase Client ใน Expo: การนำ Supabase เข้าใช้งานในโครงการ Expo SDK 54 ต้องติดตั้งไลบรารี @supabase/supabase-js (Supabase, 2024) ร่วมกับ react-native-url-polyfill เพื่อแก้ปัญหาการจัดการโครงสร้างลิงก์ และควรเก็บคีย์ความลับทั้งหมดไว้ในรูปตัวแปรสภาพแวดล้อมที่ขึ้นต้นด้วย EXPO_PUBLIC_ ภายในไฟล์ .env นอกสุดของโปรเจกต์เพื่อยกระดับความปลอดภัยก่อนเปิดใช้งานจริง

การอ่านข้อมูลจากตาราง (Read / Select Data): เมธอด .select() ใช้สำหรับการเรียกค้นหาและดึงข้อมูลจากตารางฐานข้อมูล Supabase โดยเราสามารถสืบค้นคอลัมน์ทั้งหมดแบบเจาะจง หรือต่อคำสั่งคัดกรองผลลัพธ์ผ่านตัวกรองเฉพาะ เช่น .eq() สำหรับเทียบข้อมูลค่าเท่า และ .ilike() ในการค้นหาบางส่วนของคำเพื่อส่งผลข้อมูลที่จัดเรียงถูกต้องกลับไปที่แอป

การเพิ่มข้อมูลแถวใหม่ (Create / Insert Data): การบันทึกข้อมูลใหม่ลงในตารางใช้เมธอด .insert() โดยยอมรับข้อมูลรูปแาร์เรย์ของออบเจกต์ ซึ่งสามารถอัปเดตฟิลด์ข้อมูลที่ต้องการและเรียกใช้งานคำสั่งพ่วงต่ออย่าง .select() เพื่อร้องขอให้เซิร์ฟเวอร์ส่งข้อมูลบรรทัดที่บันทึกสำเร็จล่าสุดย้อนกลับมาหาตัวแปรบนแอปทันทีเพื่อประมวลผลในขั้นตอนถัดไป

การแก้ไขและปรับปรุงข้อมูล (Update Data): กระบวนการแก้ไขค่าของข้อมูลคอลัมน์ในแถวที่เลือกจะสั่งงานผ่านเมธอด .update() โดยข้อควรระวังหลักคือผู้พัฒนาจำเป็นต้องผูกตัวกรองอย่างเช่น .eq('id', taskId) ต่อท้ายเสมอ เพื่อระบุขอบเขตการแก้ไขให้ตรงตามแถวเป้าหมายและจำกัดไม่ให้คำสั่งเข้าไปเขียนทับแถวข้อมูลอื่น ๆ ทั้งตารางข้อมูล

การลบข้อมูลออกจากตาราง (Delete Data): การถอดถอนแถวข้อมูลออกจากตารางอย่างมั่นคงถาวรทำได้โดยใช้เมธอด .delete() ซึ่งต้องใช้ร่วมกับตัวกรอง .eq() ในการระบุค่าไอดีเอกลักษณ์เพื่อกำจัดการลบแถวข้อมูลที่ไม่พึงประสงค์ โดยคำสั่งจะคืนค่ารายงานความขัดข้องกลับมาในคีย์ error เพื่อให้เราวิเคราะห์และอัปเดตสถานะลิสต์บนหน้าจอได้อย่างถูกต้อง

การจัดการสถานะการทำงานและประมวลผลข้อผิดพลาด: เนื่องจากคุณภาพสัญญาณเน็ตเวิร์กส่งผลให้เวลาในการตอบสนองฐานข้อมูลคลาวด์มีความเร็วแตกต่างกัน การออกแบบระบบจึงจำเป็นต้องครอบคลุมการจัดการสเตต loading เพื่อเปิดปิดไอคอน ActivityIndicator รวมถึงการนำเข้าโครงสร้าง try-catch มาเขียนควบคุมและใช้ response.ok หรือคีย์ error ในการตรวจจับความล้มเหลวของเครือข่ายก่อนสลับการเรนเดอร์อินเตอร์เฟซเตือนผู้ใช้งาน

แบบฝึกหัดท้ายบทเรียน

แบบทดสอบนี้มีวัตถุประสงค์เพื่อประเมินความรู้และความเข้าใจเกี่ยวกับแนวคิดระบบหลังบ้านสำเร็จรูป (BaaS), การลงทะเบียนเชื่อมต่อ Supabase Client ในสภาพแวดล้อม Expo SDK 54, การเขียนคำสั่งสืบค้นและคัดกรองข้อมูล (SELECT), การเพิ่มแถวข้อมูล (INSERT), การแก้ไขค่าข้อมูล (UPDATE), การลบข้อมูล (DELETE) และสุนทรียภาพในการจัดการข้อผิดพลาดความปลอดภัย

คำถามประจำบทเรียน (10 ข้อ)

ข้อ 1: ข้อใดคือบทบาทหน้าที่ที่ถูกต้องที่สุดของสถาปัตยกรรม Backend-as-a-Service (BaaS) เช่น Supabase?

ก. ทำการอัปเกรดความเร็วในการประมวลผล CPU และ RAM บนสมาร์ทโฟนของผู้ใช้งานโดยตรง

ข. จัดสรรระบบบริการหลังบ้านสำเร็จรูป เช่น ฐานข้อมูล ระบบยืนยันตัวตน และคลังเก็บไฟล์บนระบบคลาวด์ เพื่อให้แอปมือถือเรียกใช้งานได้ทันทีโดยไม่ต้องเขียนเซิร์ฟเวอร์เอง

ค. ทำหน้าที่เขียนสไตล์ชีทสำหรับจัดวาง Flexbox ในแบบออโต้

ง. บีบอัดขนาดโค้ดของแอปพลิเคชันให้เหลือเพียงหน้าจอเดียวชั่วคราว

ข้อ 2: ฐานข้อมูลหลักที่รันอยู่เบื้องหลังและคอยจัดสรรแถวตารางข้อมูลในคลาวด์ของ Supabase คือฐานข้อมูลประเภทใด?

ก. MongoDB (NoSQL)

ข. MySQL (Relational)

ค. PostgreSQL (Relational)

ง. Firebase Realtime Database (JSON)

ข้อ 3: การดึงคีย์ Anon Key จากระบบของ Supabase มาใช้เชื่อมต่อนั้น Anon Key มีลักษณะความปลอดภัยอย่างไร?

ก. เป็นคีย์ลับสูงสุดระดับ Root ห้ามใครพบเห็นเด็ดขาดเนื่องจากใช้ลบระบบทั้งหมดได้

ข. เป็นคีย์สาธารณะ (Public API Key) สำหรับส่งไปพร้อม API เพื่อยืนยันสิทธิ์เข้าใช้ขั้นต้นของลูกค้าร่วมกับการตั้งเงื่อนไขสิทธิ์ RLS (Row Level Security)

ค. เป็นรหัสผ่านสำหรับเข้าสู่โหมดนักพัฒนาบนแอนดรอยด์

ง. เป็นรหัสยืนยันตนสำหรับการอัปเดตแอปขึ้นไปบน Google Play Store

ข้อ 4: ใน Expo SDK 54 เมื่อต้องการกำหนดค่ารหัสโครงการและคีย์ Anon Key ไว้ในไฟล์ .env เพื่อไม่ให้โค้ดรั่วไหล ข้อใดคือรูปแบบการตั้งชื่อตัวแปรสภาพแวดล้อม (Environment Variables) ที่ถูกต้อง?

ก. `SUPABASE_URL=...`

ข. `SECRET_SUPABASE_URL=...`

ค. `EXPO_PUBLIC_SUPABASE_URL=...`

ง. `REACT_APP_SUPABASE_URL=...`

ข้อ 5: เหตุใดเราจึงจำเป็นต้องติดตั้งและสั่งนำเข้า `import 'react-native-url-polyfill/auto';` ไว้ที่ส่วนบนสุดของไฟล์กำหนดค่า Supabase Client?

ก. เพื่อบีบอัดขนาดไฟล์ภาพของโลโก้ให้เล็กลง

ข. เพื่อชดเชยและแก้ไขปัญหาที่ React Native รันไทม์ไม่มีคลาสคำสั่งจัดการ URL (URL API Class) ในระดับเนทีฟ ซึ่งทำให้ไลบรารีของ Supabase ส่งรีเควสไม่ได้

ค. เพื่อสั่งเปิดสัญญาณบลูทูธของมือถือเชื่อมเข้าคลาวด์

ง. เพื่อสลับภาษาในแบบฟอร์มให้กลายเป็นภาษาไทยอัตโนมัติ

ข้อ 6: คำสั่งสืบค้นข้อความ SQL-like ใดใน Supabase JS Client ที่ใช้สำหรับระบุชื่อตารางเป้าหมายที่ต้องการเปิดติดต่อข้อมูล?

ก. `.select()`

ข. `.from()`

ค. `.insert()`

ง. `.eq()`

ข้อ 7: หากต้องการดึงข้อมูลทุกคอลัมน์จากตารางชื่อ "tasks" โดยเรียงลำดับแถวเอาโพสต์ที่มีอายุการบันทึกล่าสุดขึ้นก่อน ข้อใดเขียนคำสั่งได้ถูกต้องและมีประสิทธิภาพที่สุด?

ก. `await supabase.from('tasks').select('*').order('created_at', { ascending: false })`

ข. `await supabase.from('tasks').select('all').eq('id', 1)`

ค. `await supabase.from('tasks').insert({ order: 'created_at' })`

ง. `await supabase.from('tasks').delete().order('created_at')`

ข้อ 8: การเพิ่มข้อมูลแถวใหม่เข้าไปในตารางด้วยเมธอด `.insert()` ของ Supabase ข้อมูลดังกล่าวต้องมีโครงสร้างของอาร์กิวเมนต์เป็นประเภทใด?

ก. ข้อความสตริงระบุคีย์ดิบ ๆ

ข. อาร์เรย์ของออบเจกต์ (Array of Objects) เช่น `[{ key: value }]`

ค. ตัวแปรประเภทตัวเลขทศนิยมเดี่ยว

ง. ฟังก์ชันประเภท Callback

ข้อ 9: พฤติกรรมการทำงานของระบบความปลอดภัย RLS (Row Level Security) ของ Supabase ข้อใดอธิบายได้ถูกต้อง?

ก. ทำหน้าที่บังคับล็อกหน้าจอโทรศัพท์มือถือหากกรอกรหัสผ่านผิดเกิน 3 ครั้ง

ข. เป็นกฎความปลอดภัยบนระบบคลาวด์เพื่ออนุญาตหรือปิดกั้นให้เฉพาะผู้ใช้ที่มีสิทธิ์เข้าถึงหรือแก้ไขเฉพาะแถวข้อมูลที่ตนเองเป็นเจ้าของเท่านั้น

ค. ใช้จำกัดความสูงความกว้างของฟอร์ม TextInput

ง. เป็นฟังก์ชันลบข้อมูลขยะออกจากหน่วยความจำ RAM ของมือถือโดยออโต้

ข้อ 10: หากต้องการแก้ไขสถานะการทำงานในคอลัมน์ `is_completed` ให้กลายเป็นค่า `true` เฉพาะแถวงานที่มีรหัสไอดี ID เท่ากับ 5 ข้อใดเขียนคำสั่งระบุเงื่อนไขได้ถูกต้องและเจาะจงเป้าหมายที่สุด?

ก. `await supabase.from('tasks').update({ is_completed: true }).eq('id', 5)`

ข. `await supabase.from('tasks').insert({ is_completed: true }).eq('id', 5)`

ค. `await supabase.from('tasks').delete().eq('id', 5)`

ง. `await supabase.from('tasks').select('is_completed').eq('id', 5)`

เฉลยคำตอบและคำอธิบายโดยละเอียด

เฉลย ข้อ 1: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: ไม่สามารถยุ่งเกี่ยวหรือยกระดับฮาร์ดแวร์จริงอย่าง CPU/RAM ของเครื่องปลายทางได้

• ข. ถูก: BaaS (เช่น Supabase) ถูกดีไซน์มาเพื่อประหยัดแรงผู้พัฒนาแอปโดยการจัดการหลังบ้าน เช่น Database และ Auth ให้สำเร็จรูปบนระบบ Cloud

• ค. และ ง. ผิด: ไม่เกี่ยวข้องกับ Flexbox และไม่มีฟังก์ชันช่วยย่อบีบอัดหน้าจอแอป

เฉลย ข้อ 2: ตอบ ค.

• เหตุผลประกอบ:

• ก. ข. และ ง. ผิด: Supabase เลือกใช้งานฐานข้อมูลประเภท Relational SQL และใช้ PostgreSQL เป็นแกนหลักขับเคลื่อนเท่านั้น

• ค. ถูก: โครงสร้างดาต้าเบสเบื้องหลังของ Supabase คือ PostgreSQL 100% ทำให้สนับสนุนฟีเจอร์อย่าง SQL, Constraints และความสัมพันธ์ข้อมูลสมบูรณ์

เฉลย ข้อ 3: ตอบ ข.

• เหตุผลประกอบ:

• ก. ผิด: คีย์สูงสุดคือ Service Role Key ซึ่งห้ามนำมาแปะลงในแอปมือถือเด็ดขาด เพราะข้ามระบบความปลอดภัยได้หมด

• **ข. ถูก:** Anon Key เป็นคีย์ระดับสาธารณะที่อนุญาตให้ลูกค้ายิง API ค้นหาตารางได้ โดยคีย์นี้จะทำงานร่วมกับระบบ RLS (Row Level Security) เพื่อจำกัดความปลอดภัยของตารางข้อมูล

• **ค. และ ง. ผิด:** ไม่ใช่รหัสสำหรับ Android หรือสิทธิ์ในการส่งแอปขึ้น Play Store

เฉลย ข้อ 4: ตอบ ค.

• **เหตุผลประกอบ:**

• **ก. ข. และ ง. ผิด:** ตัวแปรอื่นจะไม่ได้รับการดึงเข้าระบบจาวาสคริปต์อัตโนมัติในฝั่ง Expo Client

• **ค. ถูก:** ใน Expo SDK 54 ตัวแปรในไฟล์ **.env** ที่แอปจะอ่านเข้าไปรันใช้งานได้ในโค้ดดิ้งฝั่งไคลเอนต์ จำเป็นต้องมีคำนำหน้า (Prefix) เป็น **`EXPO_PUBLIC_`** เสมอ

เฉลย ข้อ 5: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไม่เกี่ยวข้องกับการย่อขนาดไฟล์รูปภาพใด ๆ

• **ข. ถูก:** รันไทม์ของ React Native ต่างจากเว็บบราวเซอร์ตรงที่ไม่มีคลาส **URL** หรือ **`URLSearchParams`** ดั้งเดิม การนำเข้าตัว polyfill จะชดเชยช่องว่างนี้ เพื่อให้ Supabase JS Client ส่ง URL รีเควสเชื่อมต่อได้สมบูรณ์โดยแอปไม่ระเบิด

• **ค. และ ง. ผิด:** ไม่สามารถควบคุมการเปิดบลูทูธเครื่อง หรือแก้ไขฟอร์มภาษาไทยได้โดยออโต้

เฉลย ข้อ 6: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** `.select()` คุมการค้นหาคอลัมน์

• **ข. ถูก:** `.from('ชื่อตาราง')` บ่งชี้เฉพาะถึงแหล่งข้อมูลตารางเป้าหมายที่ลงทะเบียนไว้ใน Supabase

• **ค. ผิด:** `.insert()` คุมการสร้างแถวข้อมูล

• **ง. ผิด:** `.eq()` ทำหน้าที่กรองค่าเงื่อนไข

เฉลย ข้อ 7: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** โครงสร้างที่สมบูรณ์คือระบุตารางด้วย `.from()`, เลือกคอลัมน์ทั้งหมดด้วย `.select('*')` และสั่งเรียงข้อมูลตามเวลาด้วย `.order('created_at', { ascending: false })`

• **ข. ค. และ ง. ผิด:** รูปแบบอื่นระบุคำสั่งไม่ถูกประเภท หรือมีพารามิเตอร์ผิดจุดประสงค์

เฉลย ข้อ 8: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ค. และ ง. ผิด:** เมธอดของ insert ปฏิเสธค่าข้อความเพลน ๆ หรือค่าทศนิยมเดี่ยว และไม่ใช่ฟังก์ชัน

• **ข. ถูก:** พารามิเตอร์ของ `.insert()` จะต้องรับค่าเป็น JSON Array เสมอ แม้ว่าเราจะยิงเพื่อสร้างเพียงแถวเดียว ก็ยังต้องส่งเป็น Array ที่มีออบเจกต์ชิ้นเดียวนอนอยู่ด้านใน

เฉลย ข้อ 9: ตอบ ข.

• **เหตุผลประกอบ:**

• **ก. ผิด:** ไม่ใช่ความปลอดภัยในการจำกัดรหัสผ่านบนฮาร์ดแวร์หน้าจอมือถือ

• **ข. ถูก:** Row Level Security (RLS) ของ PostgreSQL/Supabase ทำหน้าที่เช็คว่าแถวนี้เขียนค่านำเข้าโดยผู้ใช้คนไหน และใช้บล็อกไม่ให้คนนอกเขียนทับหรืออ่านข้อมูลข้ามบัญชีของคนอื่น

• **ค. และ ง. ผิด:** สไตล์ TextInput คุมด้วย CSS และ RAM คุมด้วยรันไทม์ระบบไม่ใช่ RLS

เฉลย ข้อ 10: ตอบ ก.

• **เหตุผลประกอบ:**

• **ก. ถูก:** เมธอด `.update()` จะยอมรับฟิลด์ใหม่ และต้องสั่งกรองระบุเฉพาะตัวด้วยเมธอด `.eq('id', 5)` เพื่อให้แก้เฉพาะไอดี 5

• **ข. ผิด:** insert ใช้สร้างแถวใหม่ ไม่ใช่แก้ไขค่า

• **ค. ผิด:** delete เป็นคำสั่งถอนลบข้อมูลไม่ใช่การเปลี่ยนสถานะ

• **ง. ผิด:** select เป็นเพียงการเรียกรับค่าเดิมมาแสดง

ปฏิบัติการท้ายบทเรียน: การสร้างแอปพลิเคชันซิงก์ข้อมูลสินค้าโปรดด้วย Supabase CRUD

ปฏิบัติการนี้จะฝึกทักษะการรวบรวมฟังก์ชันการติดต่อกับคลาวด์ดาต้าเบสระดับโลกอย่าง Supabase โดยผู้เรียนจะได้ออกแบบแอปพลิเคชัน **"สมุดจดสินค้าที่ต้องการและรายการโปรดออนไลน์ (Supabase Wishlist Manager)"** ซึ่งผู้ใช้สามารถพิมพ์บันทึกชื่อสินค้าและราคา (Create / Insert) โหลดลิสต์สินค้ามาเรนเดอร์ในแอป (Read / Select) กดจิ้มหัวใจสลับสถานะ

สินค้าโปรด (Update) และกดถอนรายการทิ้งออกจากตาราง (Delete) โดยทุกจังหวะการทำงานจะซิงก์ประสานกับตาราง SQL บนเซิร์ฟเวอร์ Supabase แบบเรียลไทม์

1. วัตถุประสงค์การเรียนรู้

1. สามารถตั้งค่าและลงทะเบียนใช้งาน `@supabase/supabase-js` ร่วมกับระบบคีย์ลับสภาพแวดล้อม `.env` ได้สำเร็จ

2. สามารถพัฒนาฟังก์ชันสำหรับค้นหาดึงข้อมูล (GET/Select) แถวจากคลาวด์มาเรนเดอร์ลงในคอมโพเนนต์ `FlatList`

3. สามารถเขียนระบบฟอร์มนำเข้าและยิงส่งข้อมูลแถวใหม่ (POST/Insert) ขึ้นสู่ตารางคลาวด์ได้ถูกต้อง

4. สามารถระบุคำสั่งแก้ไขค่าฟิลด์ตาราง (PATCH/Update) และคำสั่งลบข้อมูล (DELETE/Delete) ร่วมกับการคัดกรอง `.eq()`

5. สามารถจัดการสถานะกล่องความคืบหน้า (Loading) และรับมือข้อผิดพลาดเครือข่ายฐานข้อมูลได้ถูกต้อง

2. โจทย์และข้อกำหนดปฏิบัติการ (Requirements)

ผู้เรียนต้องสร้างโครงสร้างโปรเจกต์ประกอบด้วยไฟล์ดังต่อไปนี้:

```
โครงสร้างไฟล์ปฏิบัติการ:
├── .env                                  => บรรจุคีย์โครงการ EXPO_PUBLIC_SUPABASE_URL และ EXPO_PUBLIC_SUPABASE_ANON_KEY
├── lib/
│   └── supabase.ts                       => สร้าง Supabase Client
└── app/
    └── index.tsx                         => หน้าจอหลักควบคุมการทำงานของแอป Wishlist (CRUD Interface)
```

ข้อมูลโครงสร้างตารางหลังบ้านบน Supabase (Table Schema):

ผู้เรียนต้องไปที่ Supabase SQL Editor หรือ Table Editor เพื่อเปิดสร้างตารางชื่อ **`wishlist`** ประกอบด้วยคอลัมน์ดังนี้:

- `id` (int8 / identity): คีย์หลัก (Primary Key) สั่งสร้างรหัสอัตโนมัติ
- `title` (text): ชื่อของสินค้าที่อยากได้
- `price` (numeric): ราคาสินค้า

• `is_favorite` (boolean): สถานะความชอบพิเศษ (ค่าเริ่มต้น: `false`)

• `created_at` (timestamptz): วันเวลาบันทึก (ค่าเริ่มต้น: `now()`)

เงื่อนไขการจัดการข้อมูลในแอปพลิเคชัน:

1. **การโหลดข้อมูลเริ่มต้น (Read):** ดึงข้อมูลจากตาราง `wishlist` ทั้งหมดมาแสดง เรียงตามเวลาล่าสุด

2. **การบันทึกชิ้นใหม่ (Create):** มีฟอร์มรับอินพุตกรอกชื่อสินค้าและราคา เมื่อส่งสำเร็จให้นำผลลัพธ์ข้อมูลแถวล่าสุดที่ได้ย้อนกลับมาจาก Supabase เสียบแทรกด้านบนสุดของสเตตลิสต์ทันที

3. **การสลับหัวใจรายการโปรด (Update):** เมื่อกดแตะรูปสัญลักษณ์หัวใจ ให้ยิงอัปเดตสลับค่าบูลีนของ `is_favorite` เฉพาะไอดีนั้น ๆ บนคลาวด์

4. **การถอนรายการ (Delete):** เมื่อกดไอคอนถังขยะ ให้ลบแถวนั้นทิ้ง และรีแฟกเตอร์ฟิลเตอร์กรองแถว ID นั้นออกจากจอแอป

3. ขั้นตอนการลงมือปฏิบัติ (Step-by-Step Tutorial)

ขั้นตอนที่ 1: ติดตั้งคลังไลบรารีและสร้างระบบคีย์ลับ

สั่งรันติดตั้งคำสั่งในแอปพลิเคชัน:

```
npm install @supabase/supabase-js react-native-url-polyfill
```

จากนั้นสร้างไฟล์ `.env` ที่โฟลเดอร์นอกสุดของแอป ป้อน URL และ Anon Key ที่ได้มาจาก Supabase:

```
EXPO_PUBLIC_SUPABASE_URL=https://your-supabase-url.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-supabase-anon-key-here
```

ขั้นตอนที่ 2: ตั้งค่าโมดูล Client ใน `lib/supabase.ts`

สร้างไฟล์ `lib/supabase.ts` เพื่ออำนวยความสะดวกในการจัดแชร์ออบเจกต์คำสั่ง:

```
import 'react-native-url-polyfill/auto';
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL || '';
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY || '';

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

ขั้นตอนที่ 3: เขียนกระบวนการ CRUD เชื่อม Supabase

สร้างฟังก์ชันสำหรับทริกเกอร์เหตุการณ์ต่าง ๆ:

```
// ดึงข้อมูล Select
const fetchItems = async () => {
  const { data, error } = await supabase
  .from('wishlist')
  .select('*')
  .order('created_at', { ascending: false });
  if (!error) setWishlist(data || []);
};

// เขียนบันทึก Insert
const handleAddItem = async () => {
  const { data, error } = await supabase
  .from('wishlist')
  .insert([{ title, price: parseFloat(price), is_favorite: false }])
  .select();
  if (!error) setWishlist([data[0], ...wishlist]);
};
```

4. โค้ดเฉลยฉบับสมบูรณ์ (Complete Solution)

ผู้เรียนสามารถนำโค้ดอินเทอร์เฟซ Wishlist ฉบับสมบูรณ์ชุดนี้บันทึกใส่ลงในไฟล์ app/index.tsx เพื่อสั่งทำงานและซิงก์ฐานข้อมูลออนไลน์แบบเรียลไทม์กับโครงการ Supabase:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  TextInput,
  TouchableOpacity,
  FlatList,
  ActivityIndicator,
  Alert,
  SafeAreaView,
  Keyboard
} from 'react-native';
import { supabase } from '../lib/supabase';

// โครงสร้างประเภทข้อมูล Wishlist
interface WishItem {
  id: number;
  title: string;
  price: number;
```

```
  is_favorite: boolean;
  created_at: string;
}

export default function SupabaseWishlistManager() {
  const [wishlist, setWishlist] = useState<WishItem[]>([]);
  const [title, setTitle] = useState('');
  const [price, setPrice] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [isSubmitting, setIsSubmitting] = useState(false);

 // 1. เรียกดึงข้อมูลตารางตอนเริ่มต้นเปิดแอป
  useEffect(() => {
    fetchWishlist();
  }, []);

 // 2. ฟังก์ชัน READ: ค้นหาข้อมูลสินค้าทั้งหมด
  const fetchWishlist = async () => {
    try {
      setIsLoading(true);
      const { data, error } = await supabase
    .from('wishlist')
    .select('*')
    .order('created_at', { ascending: false });

      if (error) {
        throw error;
      }
      setWishlist(data || []);
    } catch (error: any) {
      Alert.alert('เกิดข้อผิดพลาด', error.message || 'ไม่สามารถโหลดข้อมูลคลาวด์ได้');
    } finally {
      setIsLoading(false);
    }
  };

 // 3. ฟังก์ชัน CREATE: บันทึกข้อมูลสินค้าแถวใหม่
  const handleAddItem = async () => {
    const parsedPrice = parseFloat(price);

    if (!title.trim() || isNaN(parsedPrice)) {
     Alert.alert('ข้อมูลไม่ครบ', 'กรุณาระบุชื่อสินค้าและราคาให้ถูกต้อง');
      return;
    }

    try {
      setIsSubmitting(true);
      Keyboard.dismiss();
```

```
    // ยิงข้อมูลขึ้นคลาวด์ตาราง wishlist
       const { data, error } = await supabase
     .from('wishlist')
     .insert([
           {
             title: title.trim(),
             price: parsedPrice,
             is_favorite: false
           }
     ])
     .select();

       if (error) {
         throw error;
       }

    // พ่วงข้อมูลใหม่ต่อหน้าสุดของลิสต์บนจอ
       if (data && data.length > 0) {
         setWishlist([data[0], ...wishlist]);
         setTitle('');
         setPrice('');
         Alert.alert('สำเร็จ!', 'บันทึกรายการโปรดของคุณเรียบร้อย');
       }
     } catch (error: any) {
       Alert.alert('บันทึกไม่สำเร็จ', error.message || 'เกิดข้อขัดข้องในการซิงก์ข้อมูล');
     } finally {
       setIsSubmitting(false);
     }
   };

 // 4. ฟังก์ชัน UPDATE: อัปเดตสลับสถานะความชอบสินค้าโปรด (Favorite status)
   const handleToggleFavorite = async (itemId: number, currentStatus:
boolean) => {
     try {
       const { error } = await supabase
     .from('wishlist')
     .update({ is_favorite: !currentStatus })
     .eq('id', itemId);

       if (error) {
         throw error;
       }

    // อัปเดตสเตตบนหน้าจอทันทีเพื่อแสดงสีหัวใจที่สลับตามจริง
       setWishlist(
         wishlist.map((item) =>
           item.id === itemId ? { ...item, is_favorite: !currentStatus } :
```

```
item
    )
  );
    } catch (error: any) {
      Alert.alert('แก้ไขข้อมูลล้มเหลว', error.message);
    }
  };

 // 5. ฟังก์ชัน DELETE: ถอดรายการสินค้าทิ้ง
  const handleDeleteItem = async (itemId: number) => {
    Alert.alert(
      'ยืนยันการลบ',
      'คุณแน่ใจหรือไม่ว่าต้องการนำรายการโน้ตนี้ออกถาวร?',
   [
        { text: 'ยกเลิก', style: 'cancel' },
        {
          text: 'ลบทิ้ง',
          style: 'destructive',
          onPress: async () => {
            try {
              const { error } = await supabase
        .from('wishlist')
        .delete()
        .eq('id', itemId);

              if (error) {
                throw error;
              }

        // กรองลบไอดีออกจากหน้าจอ
              setWishlist(wishlist.filter((item) => item.id !== itemId));
      } catch (error: any) {
              Alert.alert('ไม่สามารถลบข้อมูลได้', error.message);
            }
          }
        }
   ]
  );
  };

  return (
    <SafeAreaView style={styles.safeArea}>
      <View style={styles.container}>

    {/* หัวข้อแดชบอร์ด */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>🛒 Supabase Wishlist</Text>
```

```
          <Text style={styles.headerSubtitle}>จัดการความต้องการซิงก์คลาวด์
เรียลไทม์</Text>
        </View>

        {/* แบบฟอร์มเพิ่มรายการสินค้าใหม่ */}
        <View style={styles.formCard}>
          <TextInput
            style={styles.input}
            placeholder="ชื่อสินค้าที่อยากได้ (เช่น iPhone 16)..."
            placeholderTextColor="#64748b"
            value={title}
            onChangeText={setTitle}
     />
          <TextInput
            style={styles.input}
            placeholder="ระบุราคาสินค้า (บาท)..."
            placeholderTextColor="#64748b"
            value={price}
            onChangeText={setPrice}
            keyboardType="numeric"
     />

          {isSubmitting ? (
            <ActivityIndicator size="small" color="#38bdf8"
style={styles.spinner} />
     ) : (
            <TouchableOpacity style={styles.submitButton}
onPress={handleAddItem}>
              <Text style={styles.submitButtonText}>➕ บันทึกข้อมูลขึ้น
คลาวด์</Text>
            </TouchableOpacity>
     )}
        </View>

        {/* รายการแสดงผล Wishlist */}
        <View style={styles.listSection}>
        <View style={styles.listHeader}>
            <Text style={styles.listTitle}>ของที่คุณอยากได้</Text>
            <TouchableOpacity onPress={fetchWishlist}
disabled={isLoading}>
              <Text style={styles.refreshText}>{isLoading ? 'กำลังซิงก์...' :
'🔄 รีเฟรชฐานข้อมูล'}</Text>
            </TouchableOpacity>
          </View>

          {isLoading ? (
            <View style={styles.centerLoading}>
```

```
              <ActivityIndicator size="large" color="#38bdf8" />
              <Text style={styles.loadingText}>กำลังดึงข้อมูลตารางสินค้าโปรด...</Text>
            </View>
      ):(
            <FlatList
              data={wishlist}
              keyExtractor={(item) => item.id.toString()}
              renderItem={({ item }) => (
          <View style={styles.itemCard}>
                  <View style={styles.itemInfo}>
                    <Text style={styles.itemTitle}>{item.title}</Text>
                    <Text style={styles.itemPrice}>฿
{item.price.toLocaleString()}</Text>
            </View>

                  {/* ปุ่มเปลี่ยนสถานะ Favorite และปุ่มลบ */}
                  <View style={styles.actionRow}>
                    <TouchableOpacity
                      onPress={() => handleToggleFavorite(item.id,
item.is_favorite)}
                      style={styles.actionIcon}
                    >
                      <Text style={styles.heartText}>{item.is_favorite ?
'❤️' : '🤍'}</Text>
                    </TouchableOpacity>

                    <TouchableOpacity
                      onPress={() => handleDeleteItem(item.id)}
                      style={styles.actionIcon}
                    >
                      <Text style={styles.trashText}>🗑️</Text>
                    </TouchableOpacity>
            </View>
                </View>
        )}
              ListEmptyComponent={
                <View style={styles.emptyContainer}>
                  <Text style={styles.emptyText}>ไม่มีข้อมูลสินค้าในตารางของคุณ
ขณะนี้</Text>
                </View>
              }
        />
      )}
        </View>
      </View>
    </SafeAreaView>
  );
}
```

```
const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: '#0f172a',
  },
  container: {
    flex: 1,
    padding: 16,
  },
  header: {
    alignItems: 'center',
    marginVertical: 10,
  },
  headerTitle: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  headerSubtitle: {
    fontSize: 12,
    color: '#94a3b8',
    marginTop: 4,
  },
  formCard: {
    backgroundColor: '#1e293b',
    borderRadius: 16,
    padding: 16,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 20,
  },
  input: {
    backgroundColor: '#0f172a',
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#334155',
   color: '#ffffff',
    padding: 12,
    fontSize: 14,
    marginBottom: 10,
  },
  submitButton: {
    backgroundColor: '#38bdf8',
    borderRadius: 8,
    height: 44,
    justifyContent: 'center',
```

```
    alignItems: 'center',
    marginTop: 4,
  },
  submitButtonText: {
    color: '#0f172a',
    fontWeight: 'bold',
    fontSize: 14,
  },
  spinner: {
    paddingVertical: 10,
  },
  listSection: {
    flex: 1,
  },
  listHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    marginBottom: 12,
  },
  listTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#ffffff',
  },
  refreshText: {
    color: '#38bdf8',
    fontSize: 13,
  },
  centerLoading: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  loadingText: {
    color: '#94a3b8',
    marginTop: 8,
    fontSize: 13,
  },
  itemCard: {
    flexDirection: 'row',
    backgroundColor: '#1e293b',
    borderRadius: 12,
    padding: 14,
    borderWidth: 1,
    borderColor: '#334155',
    marginBottom: 10,
```

```
    alignItems: 'center',
    justifyContent: 'space-between',
  },
  itemInfo: {
    flex: 1,
  },
  itemTitle: {
    color: '#ffffff',
    fontWeight: 'bold',
    fontSize: 15,
    textTransform: 'capitalize',
  },
  itemPrice: {
    color: '#10b981',
    fontWeight: 'bold',
    fontSize: 13,
    marginTop: 4,
  },
  actionRow: {
    flexDirection: 'row',
    alignItems: 'center',
  },
  actionIcon: {
    marginLeft: 16,
    padding: 6,
  },
  heartText: {
    fontSize: 18,
  },
  trashText: {
    fontSize: 16,
  },
  emptyContainer: {
    alignItems: 'center',
    paddingVertical: 40,
  },
  emptyText: {
    color: '#64748b',
    fontSize: 14,
  },
});
```

เอกสารอ้างอิง

Supabase. (2024). *React Native Supabase Quickstart*. จาก https://supabase.com/docs/guides/getting-started/tutorials/with-react-native

Supabase. (2024). *Supabase Database Quickstart*. จาก https://supabase.com/docs/guides/database/overview

Supabase. (2024). *Supabase JavaScript Client Database API*. จาก https://supabase.com/docs/reference/javascript/select